BRONZE	SILVER	GOLD

Proyecto PySpark

Arquitectura Medallón en Lakehouse

TP 6 – Implementación en Microsoft Fabric

Aprendiz	Diego Euler
Programa	**Get Talent v4 – Data Engineering**
Entorno	Workspace: gt-deuler-dev
Plataforma	Microsoft Fabric
Fecha de Entrega	19 de Febrero de 2026

Tabla de Contenidos

1. Introducción 3
   1.1 Objetivo del Pipeline 3
   1.2 Herramientas y Tecnologias Utilizadas 3
2. Arquitectura del Pipeline 4
   2.1 Diagrama de Arquitectura 4
   2.2 Descripcion por Capa 4
      Capa Bronze — Ingestión Raw 4
      Capa Silver — Transformaciones y Enriquecimiento 5
      Capa Gold — One Big Table (OBT) 5
3. Transformaciones Capa Silver 6
   3.1 Tabla de Transformaciones por Dataset 6
   3.2 Justificacion de Transformaciones 6
      shop_carrito — Dataset Principal 6
      shop_clientes — Datos de Clientes 7
      Otros Datasets 7
4. Capa Gold — One Big Table (OBT) 9
   4.1 Concepto de OBT 9
   4.2 Estrategia de Construccion 9
   4.3 Detalle de Joins 9
   4.4 Validaciones Realizadas 10
5. Delta Lake — Justificación Técnica 11
   5.1 Ventajas ACID 11
   5.2 Versionado y Time Travel 11
   5.3 Performance 11
6. Mejoras Propuestas para Silver (Opcional) 12
   6.1 Slowly Changing Dimensions (SCD Tipo 2) 12
   6.2 Validaciones de Calidad (Data Quality Framework) 12
   6.3 Z-Order y Optimizacion de Particiones 12
   6.4 Procesamiento Incremental 12
   6.5 Auditoria de Cambios 13
7. Conclusiones 14
   7.1 Cumplimiento de Requisitos 14

1. Introducción

El presente documento describe la implementación completa del pipeline de datos desarrollado para el Trabajo Practico 6 del programa Get Talent v4. El proyecto aplica la Arquitectura Medallón (Bronze–Silver–Gold) sobre Microsoft Fabric, utilizando PySpark como motor de procesamiento distribuido y OneLake como capa de almacenamiento unificada.

La arquitectura Medallón es un patrón de diseño para data lakes que organiza los datos en capas progresivas de calidad y valor analítico. Cada capa tiene una responsabilidad especifica dentro del ciclo de vida del dato, garantizando trazabilidad, reproducibilidad y eficiencia en el procesamiento.

1.1 Objetivo del Pipeline

El pipeline tiene como objetivo principal tomar datos crudos en formato CSV desde el origen lh_origin_get_talent, ingerirlos, transformarlos y enriquecerlos progresivamente hasta construir una tabla analitica final (OBT) lista para consumo por equipos de Business Intelligence y análisis.

Alcance del Proyecto
7 datasets de la carpeta shop_carrito | 3 notebooks independientes (Bronze / Silver / Gold) | Workspace: gt-deuler-dev | Lakehouses: lh_bronze, lh_silver, lh_gold | Mas de 120,000 registros procesados

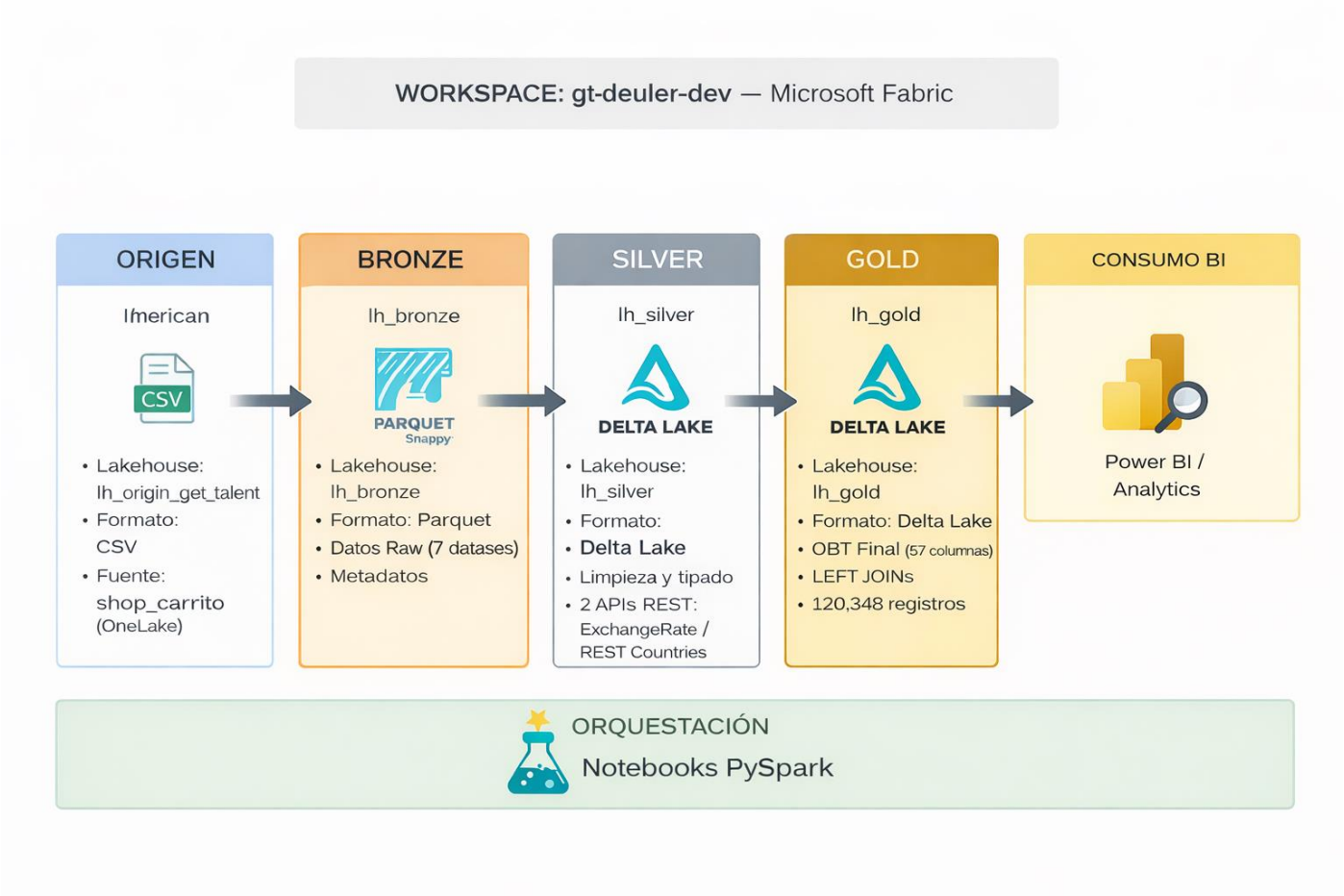
1.2 Herramientas y Tecnologias Utilizadas

- Microsoft Fabric – Plataforma cloud unificada para Data Engineering y analítica
- OneLake – Almacenamiento centralizado con arquitectura multi-Lakehouse
- PySpark – Motor de procesamiento distribuido para transformaciones a escala
- Delta Lake – Formato de tabla con soporte ACID, versionado y time travel
- Apache Parquet – Formato columnar para ingestion en capa Bronze
- ExchangeRate-API – API publica para conversion de tipos de cambio en tiempo real
- REST Countries API – API publica para enriquecimiento de datos geograficos de paises
- Python requests – Libreria para consumo de APIs REST desde notebooks

2. Arquitectura del Pipeline

La arquitectura implementada sigue el patrón Medallón, ampliamente adoptado en implementaciones modernas de Data Lakehouse. Este patrón define tres zonas de calidad progresiva: Bronze (datos crudos), Silver (datos limpios y enriquecidos) y Gold (datos modelados para consumo analítico).

2.1 Diagrama de Arquitectura



2.2 Descripción por Capa

Capa Bronze — Ingestión Raw

Responsabilidad: Preservar los datos tal como provienen del origen, sin modificaciones de lógica de negocio. La capa Bronze actúa como un "lago histórico" que garantiza trazabilidad y reproducibilidad.

- Origen: lh_origin_get_talent / Files / shop_carrito (7 CSV con delimitador ';')
- Destino: lh_bronze / Files / bronze / shop_carrito
- Formato: Apache Parquet con compresion Snappy
- Modo de escritura: Overwrite (idempotente)
- Metadatos agregados: _bronze_ingestion_timestamp, _bronze_source_file, _bronze_dataset
- Procesamiento: Un directorio Parquet independiente por cada dataset

Capa Silver — Transformaciones y Enriquecimiento

Responsabilidad: Aplicar transformaciones de calidad, limpieza, tipado correcto y enriquecimiento con fuentes externas. La capa Silver es la base analitica confiable del pipeline.

- Origen: lh_bronze / Files / bronze / shop_carrito
- Destino: lh_silver / Files / silver / shop_carrito
- Formato: Delta Lake (con transaction log _delta_log)
- Transformaciones: Minimo 2 por dataset (casting, deduplicacion, derivacion de columnas, segmentacion)
- Enriquecimiento API: 2 datasets con integracion REST (shop_carrito y shop_clientes)
- Una tabla Delta independiente por dataset

Capa Gold — One Big Table (OBT)

Responsabilidad: Construir una tabla analitica unificada (OBT) que consolide toda la informacion relevante para el consumo analitico. La Gold Layer es el destino final para herramientas de BI y data science.

- Origen: lh_silver / Files / silver / shop_carrito (7 tablas Delta)
- Destino: lh_gold / Files / gold / obt_shop_carrito
- Formato: Delta Lake con compresion Snappy
- Estrategia: shop_carrito como tabla base + 5 LEFT JOINs
- Resultado: 120,348 registros | 57 columnas | NULLs preservados (comportamiento esperado en OBT)

3. Transformaciones Capa Silver

La capa Silver es el corazón del pipeline. Cada uno de los 7 datasets de Bronze es leido, transformado con minimo 2 operaciones justificadas, y guardado como tabla Delta independiente. Dos datasets adicionales son enriquecidos mediante integracion con APIs REST publicas.

3.1 Tabla de Transformaciones por Dataset

Dataset	Transformaciones Aplicadas	API Utilizada	Registros
shop_carrito	Cast de tipos, dropDuplicates, parseo de fechas, normaliz. strings, columnas derivadas (anio/mes/dia/hora), segmento_monto, conversion multi-moneda	**ExchangeRate-API (USD/ARS/EUR/BRL)**	**120,348**
shop_clientes	Normaliz. nombre/email/telefono, concat nombre_completo, upper/trim, segmento_provincia, sexo_desc, enriquecimiento pais	**REST Countries API (nombre, region, capital, moneda)**	**115,544**
articulos	Parsing de columna unica, extraccion de campos, cast tipos, categoria_costo, costo_con_margen, estado_desc	—	**58**
campanias_gp	Parsing raw data, parseo timestamps, estado_desc, anio/mes creacion	—	**96**
shop_carrito_estados	upper/trim, cast interno, categoria_estado (INTERNO/EXTERNO)	—	**16**
shop_carrito_estados_pago	upper/trim, clasificacion nivel_riesgo (BAJO/MEDIO/ALTO)	—	**5**
shop_formapago	upper/trim, tipo_medio (TARJETA/DEBITO/TRANSFERENCIA/etc.)	—	**12**

3.2 Justificacion de Transformaciones

shop_carrito — Dataset Principal

Transformacion 1 – Limpieza y normalizacion de tipos:

- dropDuplicates(['id']): Elimina registros duplicados basados en la clave primaria para garantizar integridad de datos.
- to_timestamp(): Parsea el campo fecha desde formato string 'M/d/yyyy HH:mm' a tipo nativo timestamp, habilitando funciones temporales nativas de Spark.
- cast a tipos numericos: Garantiza que campos como monto, estado, id_cliente sean del tipo correcto para operaciones analiticas y joins eficientes.
- upper/trim/coalesce en localidad y provincia: Normaliza strings para evitar duplicados por diferencias de formato (ej: 'santiago' vs 'SANTIAGO').
- filter(monto >= 0): Descarta registros con montos negativos que representan datos invalidos de negocio.

Transformacion 2 – Columnas derivadas y segmentacion:

- Extraccion de anio, mes, dia, hora: Permite analisis temporal granular sin recalcular en cada consulta.
- segmento_monto (BAJO/MEDIO/ALTO/PREMIUM): Clasifica transacciones en rangos de valor para facilitar segmentacion comercial y reporting ejecutivo.

Transformacion 3 – Enriquecimiento con ExchangeRate-API:

- La API retorna tasas de cambio vigentes al momento de ejecucion del pipeline. Se agregan columnas monto_usd, monto_eur, monto_brl para habilitar analisis multi-moneda.
- Justificacion de negocio: El comercio opera en Chile (CLP) pero puede necesitar reportes financieros en dolares o euros para consolidacion regional.

shop_clientes — Datos de Clientes

Transformacion 1 – Normalizacion de campos:

- nombre_completo = concat(nombre, apellido): Genera un campo derivado de uso frecuente en reportes, evitando la concatenacion en cada consulta.
- email_limpio = lower(trim(email)): Normaliza emails para evitar duplicados por mayusculas o espacios.
- telefono_limpio = regexp_replace: Elimina caracteres no numericos del telefono para estandarizar el campo.

Transformación 2 – Segmentación geográfica:

- segmento_provincia: Agrupa provincias en categorías CENTRO/URBANO/REGIONAL para facilitar analisis de cobertura geográfica.
- sexo_desc: Traduce códigos de una letra (M/F) a descripciones legibles para reportes.

Transformación 3 – Enriquecimiento con REST Countries API:

- Consulta la API con el nombre del pais del cliente y agrega: nombre oficial, region geografica, capital, poblacion y moneda oficial.
- Justificacion de negocio: Permite analisis de clientes por region geografica y contexto economico del pais de origen.

Otros Datasets

articulos: El CSV usaba coma como delimitador, pero fue leído con ';', resultando en una columna única. Se aplico parsing via split(',') para extraer campos individuales, seguido de categorización de costo y cálculo de precio sugerido con margen del 30%.

campanias_gp: Similar al caso de articulos, requirio parsing de datos crudos. Se agrego clasificacion de estado activo/inactivo y extraccion de dimensiones temporales de creacion.

shop_carrito_estados: Normalización de strings y categorización en INTERNO/EXTERNO segun el flag 'interno'. Util para filtrar vistas de estado en la interfaz de usuario vs reportes internos.

shop_carrito_estados_pago: Clasificación de nivel de riesgo financiero por estado de pago (BAJO para aprobados, MEDIO para pendientes, ALTO para rechazados). Insumo directo para modelos de fraud scoring.

shop_formapago: Normalización y categorización por tipo de medio (TARJETA/DEBITO/TRANSFERENCIA/BILLETERA_DIGITAL/EFECTIVO). Clave para analisis de preferencias de pago y costos de procesamiento.

4. Capa Gold — One Big Table (OBT)

4.1 Concepto de OBT

Una One Big Table (OBT) es un patrón de modelado de datos para analítica que desnormaliza múltiples tablas relacionales en una única tabla ancha. Contrario al modelo en estrella tradicional, la OBT consolida toda la información relevante en una sola estructura, simplificando drásticamente las consultas de los consumidores finales (analistas, herramientas BI, modelos ML).

¿Por qué usar OBT en Gold?
Simplicidad de consulta: Los consumidores no necesitan escribir JOINs complejos. | Performance: Spark puede leer una sola tabla en paralelo sin coordinación entre tablas. | Flexibilidad: Analysts pueden filtrar, agregar y pivotear sin conocer el modelo relacional subyacente.

4.2 Estrategia de Construccion

La OBT fue construida usando shop_carrito como tabla base (la de mayor volumetria con 120,348 registros) y aplicando LEFT JOINs con todas las tablas de dimensión. Esta estrategia garantiza que:

1. Todos los registros de la tabla base se preservan en el resultado final.
2. Los registros de dimensiones sin match generan NULLs (comportamiento esperado y valido en OBT).
3. La cantidad de registros finales sea siempre >= al dataset más grande (requisito del ejercicio cumplido).
4. No se produce explosión de registros que ocurriría con INNER JOIN o CROSS JOIN.

4.3 Detalle de Joins

Tabla	Tipo Join	Clave de Union	Registros	Columnas Aportadas
shop_carrito	**BASE (tabla principal)**	—	120,348	20 columnas transaccionales
shop_clientes	LEFT JOIN	id_cliente = cliente_id	115,544	17 columnas de cliente
shop_carrito_estados	LEFT JOIN	carrito_estado_id = estado_carrito_id	16	4 columnas de estado
shop_carrito_estados_pago	LEFT JOIN	carrito_estado_pago_id = estado_pago_id	5	3 columnas de estado pago
shop_formapago	LEFT JOIN	id_fpago = forma_pago_id	12	3 columnas de pago
campanias_gp	LEFT JOIN	id_campania = campania_id	96	8 columnas de campania
articulos	NO JOINEABLE	Sin FK directa en carrito	58	—

Nota sobre artículos: La tabla artículos no fue incluida en los joins porque el schema de shop_carrito no contiene una FK directa con granularidad de item. El campo id_articulo_ppal existe pero no garantiza una

relacion limpia de 1:1 con la tabla articulos. En un pipeline productivo, existiria una tabla intermedia shop_carrito_items que relacionaria ordenes con articulos.

4.4 Validaciones Realizadas

- Conteo post-escritura: 120,348 registros escritos = 120,348 registros leidos (consistencia confirmada).
- Cumplimiento de minimo: OBT (120,348) >= dataset mas grande (120,348). Requisito cumplido.
- NULLs preservados: No se aplicaron reemplazos de NULL. El 59.4% de carrito_fecha es NULL por registros sin fecha, lo cual es un dato valido de negocio.
- Estructura delta: Existencia de _delta_log y _commits confirmada en Microsoft Fabric Explorer.

Resultado Final OBT

Tabla: obt_shop_carrito | Lakehouse: lh_gold | Formato: Delta Lake (Snappy) | Registros: 120,348 | Columnas: 57 | Workspace: gt-deuler-dev

5. Delta Lake — Justificación Técnica

Delta Lake fue adoptado como formato de almacenamiento para las capas Silver y Gold. Esta decisión técnica responde a características clave del formato que lo posicionan como estandar de facto en arquitecturas modernas de Data Lakehouse.

5.1 Ventajas ACID

Delta Lake garantiza transacciones ACID (Atomicidad, Consistencia, Isolation, Durabilidad) sobre archivos en almacenamiento de objetos como OneLake:

- Atomicidad: Una escritura o existe completa o no existe. No hay estados intermedios corruptos.
- Consistencia: El schema es validado antes de cada escritura, previniendo inconsistencias de tipo.
- Isolation: Lecturas concurrentes no son afectadas por escrituras en curso.
- Durabilidad: Una vez confirmada la transacción, los datos persisten independientemente de fallos.

5.2 Versionado y Time Travel

Cada escritura en Delta genera una nueva versión en el _delta_log. Esto habilita:

- Auditoria: Se puede consultar el historial completo de operaciones sobre la tabla.
- Rollback: Es posible revertir a cualquier versión anterior ante errores de procesamiento.
- Time Travel: Permite consultar el estado de los datos en cualquier punto del tiempo pasado.

5.3 Performance

- Z-Ordering: Permite co-ubicar datos relacionados para acelerar queries con filtros específicos.
- Data Skipping: Delta mantiene estadísticas min/max por archivo para evitar escaneos innecesarios.
- Compaction: OPTIMIZE agrupa archivos pequeños en archivos mas grandes, mejorando el throughput de lectura.
- Schema Evolution: Permite agregar columnas sin reescribir toda la tabla.

6. Mejoras Propuestas para Silver (Opcional)

Las siguientes mejoras elevarían el pipeline de un nivel académico a un nivel productivo Enterprise. Se priorizan según impacto técnico y valor de negocio:

6.1 Slowly Changing Dimensions (SCD Tipo 2)

Actualmente, la capa Silver aplica overwrite completo en cada ejecución. En un entorno productivo, los cambios en dimensiones como shop_clientes deberian ser tratados con SCD Tipo 2 para preservar el historial de cambios:

- Agregar columnas: valid_from (timestamp inicio), valid_to (timestamp fin), is_current (boolean).
- Al detectar cambios en atributos criticos (email, direccion), cerrar el registro actual y crear uno nuevo activo.
- Implementacion con DeltaTable.merge() usando la condicion matchedUpdate / notMatchedInsert.
- Beneficio: Permite analisis historico de clientes (cuantos cambiaron de direccion, evolucion de segmento, etc.).

6.2 Validaciones de Calidad (Data Quality Framework)

Implementar un framework minimo de validacion antes de escribir en Silver:

- Completitud: Verificar que columnas criticas (id, monto, id_cliente) no superen un umbral de NULLs.
- Unicidad: Validar que no existan duplicados en claves primarias post-transformacion.
- Rangos validos: Montos entre 0 y un valor maximo razonable, fechas dentro de un rango esperado.
- Referential integrity: Verificar que los id_cliente en shop_carrito existan en shop_clientes.
- Implementacion sugerida: Great Expectations o validaciones nativas con assert sobre conteos.

6.3 Z-Order y Optimizacion de Particiones

Para mejorar la performance de lectura en tablas Silver de alta volumetria:

- Z-Order por columnas de filtro frecuente: anio, mes, estado, segmento_monto en shop_carrito.
- Particionado por fecha: partitionBy('anio', 'mes') para habilitar partition pruning en queries temporales.
- OPTIMIZE periodico: Programar OPTIMIZE ZORDER en el pipeline de mantenimiento para compactar archivos fragmentados por cargas incrementales.

6.4 Procesamiento Incremental

Reemplazar el modo overwrite por procesamiento incremental para reducir costos computacionales:

- Watermark: Registrar el ultimo timestamp procesado en una tabla de control.
- DeltaTable.merge() (UPSERT): Insertar registros nuevos y actualizar los modificados sin reescribir toda la tabla.
- Beneficio: En pipelines productivos con millones de registros diarios, el procesamiento incremental reduce el tiempo de ejecucion en un 80-90% comparado con full overwrite.

6.5 Auditoria de Cambios

Agregar tabla de auditoria del pipeline:

- Registrar por cada ejecución: dataset, timestamp_inicio, timestamp_fin, registros_leidos, registros_escritos, errores.
- Detectar automáticamente anomalías de volumen (caida mayor al 10% de registros esperados).
- Implementacion: Tabla Delta lh_bronze.audit_log o lh_silver.pipeline_audit.

7. Conclusiones

El pipeline implementado cubre exitosamente los requerimientos del TP6, aplicando la Arquitectura Medallón sobre Microsoft Fabric con PySpark como motor de procesamiento. Los puntos clave del trabajo son:

7.1 Cumplimiento de Requisitos

- Ejercicio 1: Notebooks creadas con Lakehouse lh_origin_get_talent como fuente de datos.
- Ejercicio 2: Los 7 datasets CSV convertidos a Parquet con metadatos en lh_bronze. Evidenciado en Microsoft Fabric Explorer.
- Ejercicio 3: 7 tablas Delta en lh_silver con minimo 2 transformaciones cada una y 2 APIs REST integradas.
- Ejercicio 4: OBT con 120,348 registros (>= dataset mayor), 57 columnas, NULLs preservados, formato Delta en lh_gold.
- Ejercicio 5: Documentación formal completa (presente documento).
- Ejercicio 6: Mejoras propuestas para Silver documentadas en la sección anterior.
- Ejercicio 7: Diagrama de arquitectura incluido en la sección 2.

Estado Final del Pipeline

Bronze: 7 datasets | 351,139 registros totales procesados | Silver: 7 tablas Delta | 236,079 registros | 2 APIs integradas | Gold: 1 OBT | 120,348 registros | 57 columnas | 3 notebooks entregadas